PRÁCTICA 5

"Simulación y Filtrado de Ruidos en Imágenes"

Objetivo: simulación y filtrado de ruidos en imágenes; evaluación de distintos tipos y opciones de filtrado.

La práctica se ha dividido en tres partes:

1. **Primera parte:** simulación de ruidos tipo gaussiano y *sal y pimienta*.
2. **Segunda parte:** implementación de filtros gaussiano, mediana y adaptativo.
3. **Tercera parte:** evaluación de eficiencia de filtros gaussiano, mediana y adaptativo.

PRIMERA PARTE: Simulación de ruidos.

1. Lee la imagen en escala de gris "*img.png*".

2. Corrompe la imagen anterior con ruido de tipo de sal y pimienta y de tipo gaussiano para generar imágenes ruidosas, tal y como se describe a continuación:

   a. Sal y pimienta, con $p = 0.9$ y $q = 0.95$ (ver ecuación). La imagen corrompida con este ruido (imagen A) se calculará de la siguiente manera:

   $$A(i,j) = \begin{Bmatrix} I(i,j) & x < p \\ 0 & p \le x < q \\ 255 & q \le x < 1 \end{Bmatrix}$$ siendo I la imagen original sin corromper, x una variable aleatoria uniforme de rango (0,1), q-p el porcentaje de píxeles con ruido de tipo pimienta y 1-q el porcentaje de píxeles con ruido de tipo sal.

   b. Gaussiano de media nula y desviación típica 5, 10, 35 y 50.

   Apunte teórico:

   Sea $N(0,1)$ una función distribución de probabilidad normal o gaussiana de media cero y varianza 1 ($N(0,1)$). Utilizando esta distribución, se puede obtener otra distribución gaussiana de media μ y varianza σ^2 ($N(\mu,\sigma^2)$):

   $$N(0,1) = \frac{N(\mu,\sigma^2) - \mu}{\sigma} \quad \rightarrow \quad N(\mu,\sigma^2) = \mu + \sigma N(0,1)$$

3. Visualiza la imagen original y las 5 imágenes ruidosas generadas (en una misma ventana gráfica).

4. Representa en un mismo gráfico la variación de los niveles de gris a lo largo de la línea horizontal central para la imagen original, para la imagen contaminada con ruido de tipo sal y pimienta, y para la imagen corrompida con ruido gaussiano de desviación típica 10. Observa las distintas distribuciones del ruido.

5. En un directorio de *Imágenes Generadas*, almacena la imagen corrompida con ruido sal y pimienta y las cuatro imágenes corrompidas con ruidos gaussiano (media nula y desviaciones típicas 5, 10, 35 y 50).

Objetivo teórico: conocer los dos tipos de ruido, su forma de manifestarse en imágenes digitales y saber cómo se pueden simular e implementar.

SEGUNDA PARTE: Implementación de filtros gaussiano, mediana y adaptativo.

Esta segunda parte de la práctica se debe realizar sobre la imagen corrompida con ruido sal y pimienta generada en la primera parte de la práctica

2.1.-Filtro Gaussiano

a. Implementación de *kernel* de convolución de un filtro gaussiano (este apartado debe realizarse en un cuaderno *Jupyter* independiente):

- Crea la máscara del filtro gaussiano 5x5, considerando la relación W = 5 σ (siendo σ la desviación típica del filtro y W sus dimensiones), utilizando el siguiente código (programación vectorizada sin bucles):

```
W = 5; sigma = W/5
mitad = int(W//2)
X = np.arange(-mitad, mitad + 1)
Y = np.arange(-mitad, mitad + 1)
X2, Y2 = np.meshgrid(X, Y)
hg = np.exp(-0.5 * (X2**2 + Y2**2) / (sigma**2))
hg = hg / np.sum(hg)
```

- Crea la misma máscara, pero implementando un doble bucle *for* para generar cada uno de sus valores.

- Utiliza la función de OpenCV `getGaussianKernel` para generar los valores de la máscara y comprueba que se obtienen los mismos resultados en los tres casos (pueden asumirse diferencias máximas del orden de 10^(-8)).

  Observación:

  getGaussianKernel devuelve un vector columna; se debe multiplicar este vector por su traspuesto para obtener la máscara 2D:

  ```
  gk = cv2.getGaussianKernel(W, sigma); hg = gk @ gk.T
  ```

- Visualiza la máscara en una representación de superficie 3D utilizando el siguiente código:

```
W = hg.shape[0]
mitad = W//2
```

```
X = np.arange(-mitad, mitad + 1); Y = np.arange(-mitad, mitad + 1)
X2, Y2 = np.meshgrid(X, Y)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X2, Y2, hg, cmap='viridis')
ax.set_title('Máscara Gaussiana 3D')
plt.show()
```

b. Utilizando *padding* de ceros, aplica un filtro de media simple *9x9* y un filtro gaussiano de la misma vecindad (se debe utilizar la función *getGaussianKernel* para la obtención del *kernel* de convolución) sobre la imagen corrompida con ruido sal y pimienta. Visualiza los resultados (imagen de entrada y dos imágenes filtradas) en una misma ventana gráfica. Analiza las imágenes.

c. Aplica el filtro gaussiano del apartado anterior con la función de OpenCV *GaussianBlurr* y compara el tiempo computacional y resultados de ambos procedimientos. La comparación se debe hacer con las variables de salida tipo *float32* (se asumen diferencias del orden de 10^(-5)).

2.2.- Filtro de la mediana

a. Implementación de la función *filtro_mediana*: tomando como base la programación de las funciones *filtro_máximos* y *filtro_mínimos*, implementa la función *filtro_mediana* e incorpórala en el fichero Python de funciones de filtro de orden, *funciones_filtro_orden.py*.

b. Utilizando *padding 'replicate'* y una vecindad completa 3x3, aplica un filtro de la mediana a la imagen corrompida con ruido sal y pimienta. Debe aplicarse la función *filtro_mediana* y la función de OpenCV *medianBlur* y contrastar numéricamente que se obtienen los mismos resultados.

   Observación: la función *medianBlur* recibe como entradas una imagen de entrada (*uint8* o *uint16*) en escala de grises o color *BGR* y un parámetro tipo *int* que indica la vecindad del filtro (vecindad cuadrada); como salida se genera la imagen filtrada del mismo tipo y dimensión que la imagen de entrada. El filtro opera con un *padding* "*replicate*".

c. Visualiza la imagen de entrada e imagen filtrada.

2.3.- Filtro adaptativo

a. Implementa las funciones que realizan un filtrado adaptativo sobre una imagen de entrada, *filtro_adaptativo_v1 y filtro_adaptativo_v2,* y guárdalas en el fichero Python *funciones_filtro_adapatativo.py*. Las entradas de ambas funciones deben ser:

   - *I*: *np.ndarray*, imagen de entrada 2D.
   - *V*: *np.ndarray*, matriz de vecindad (de dimensiones impares), compuesta por 0s y 1s. Solo los píxeles donde *V=1* se usan para calcular la media y desviación típica local.
   - *VarRuido*: varianza del ruido.
   - *opcionrelleno*: *string*, que indica el tipo de padding: *'replicate'*, *'reflect'* o *'zeros'* (por defecto 'reflect').
   - La función devuelve *Io*, *np.ndarray* tipo *float64*, matriz resultante tras aplicar el filtro.

Observaciones de implementación:

   - La versión 1 de la función debe implementarse recorriendo píxel a píxel la imagen.
   - La versión 2 debe implementarse de forma matricial. Para ello, se facilita la función *stdfilt*, que calcula la desviación típica local (sobre una vecindad dada) de cada punto de una matriz de entrada.
   - Todas las operaciones en ambas funciones se deben realizar en *float64*.
   - Para evitar desbordamientos en los valores resultantes de la expresión que implementa el filtrado, los valores del factor *VarRuido/VarLocal* (varianza local) deben limitarse a un rango de variación 0-1.
   - Para prevenir la división por cero, en caso de desviación típica local cero se debe sustituir su valor por el menor número positivo representable: *np.finfo(np.float64).eps*

b. Utilizando padding '*reflect'*, aplica un filtrado adaptativo 9x9 sobre la imagen corrompida con ruido sal y pimienta utilizando ambas versiones de la función. La varianza del ruido de entrada de las funciones debe ser la estimación de la varianza del ruido global presente en la imagen (debe realizarse a partir de la imagen sin corromper y

la imagen con ruido sal y pimienta). Comprueba numéricamente que ambas funciones obtienen el mismo resultado (diferencias del orden de 10^(-5)).

c. Visualiza y analiza el resultado del proceso de filtrado: imagen de entrada e imagen filtrada.

Objetivos teóricos:

- Clasificación de tipos de filtrado para reducir ruido.
- Conocer los filtros representativos de cada tipología y saber sus fundamentos de operación e implementación.

TERCERA PARTE: Evaluación de eficiencia de filtros gaussiano, mediana y adaptativo.

Esta tercera parte de la práctica se debe realizar sobre las imágenes corrompidas con ruido gaussiano de desviaciones 5, 10, 35 y 50.

a. Filtrado de imágenes: filtra las imágenes con ruido gaussiano generadas en la práctica (desviaciones 5, 10, 35 y 50), con filtros gaussiano, mediana y adaptativo, considerando *padding replicate* y vecindades cuadradas 3x3, 7x7 y 15x15 para cada filtro.

b. Evaluación de los procesos de filtrado: evalúa la eficiencia de cada proceso de filtrado midiendo la relación señal-ruido (ISNR), definida como:

$$\mathrm{ISNR}_{\mathrm{dB}} = 10 \log_{10} \frac{\sum_{i=1}^{M} \sum_{j=1}^{N} \left(\mathbf{I}(i,j) - \mathbf{G}(i,j)\right)^2}{\sum_{i=1}^{M} \sum_{j=1}^{N} \left(\mathbf{I}(i,j) - \mathbf{I_e}(i,j)\right)^2}$$

donde I es la imagen original sin ruido, G la imagen ruidosa, e I_e es la imagen filtrada. Presenta los resultados de ISNR en una tabla con el siguiente formato:

SigmaRuido	TamagnoVentana	FiltroGauss	FiltroMediana	FiltroAdaptativo
5	3			
5	7			
5	15			
10	3			
10	7			
10	15			
35	3			
35	7			
35	15			
50	3			
50	7			
50	15			

Analiza los resultados obtenidos valorando los filtros y vecindades que resultan más adecuados aplicar dependiendo de la magnitud del ruido gaussiano presente en la imagen.

c. Visualización de un determinado proceso de filtrado: definiendo la imagen ruidosa (por la desviación típica del ruido, 5, 10, 35, 50), el tipo de filtro (‘gaussiano’, ’mediana’, ’adaptativo’) y su vecindad de trabajo (3, 7, 15), se debe aplicar el filtro y visualizar en una ventana la imagen libre de ruido, la imagen ruidosa seleccionada y la imagen filtrada.

Objetivos teóricos:

- Conocer la métrica utilizada para evaluar el rendimiento de un proceso de filtrado.
- Capacidad de análisis de resultados de los diferentes procesos de filtrado ante factores representativos como la magnitud del ruido presente en las imágenes y la vecindad con la que trabaja el filtro.